

Dossier Bloc 2 RNCP39583 — Alcide

Concevoir et développer des applications logicielles

Version candidate locale : 0.13.0-rc.1 — 2026-07-20

Statut : à figer après recette, CI, audit accessibilité et déploiement du SHA final

1. Cadre officiel et règle de validation

Le présent dossier répond au référentiel RNCP39583 et non à une grille interne au projet. L'épreuve demande le code source, la documentation associée et un dossier écrit de 30 pages maximum. Les pièces attendues couvrent notamment CI, CD, architecture, prototype, tests unitaires, sécurité, accessibilité, versions, recette, correction des bogues et trois manuels d'exploitation.

Le bloc comporte neuf compétences. Il faut en acquérir au moins cinq et les quatre compétences éliminatoires doivent toutes être acquises :

- C2.2.1 — prototype ;
- C2.2.2 — harnais de tests unitaires couvrant la majorité du code développé ;
- C2.2.3 — logiciel évolutif, sécurisé, accessible et conforme ;
- C2.3.1 — cahier de recettes couvrant l'ensemble des fonctionnalités attendues.

Il n'existe pas de « seuil RNCP de 70 % ». Les seuils Vitest sont des gates de projet ; la réponse à C2.2.2 doit s'appuyer sur le périmètre réellement instrumenté et sur la représentativité des tests.

2. Synthèse de la version candidate

Compétence	État de la version candidate	Condition de fermeture
C2.1.1	Environnements décrits ; mesures finales à produire	rapport performance et preuves datées
C2.1.2	CI renforcée dans le code	premier run vert du SHA final
C2.2.1	Prototype existant et déployé	recette desktop/mobile après corrections
C2.2.2	Mesures locales API/Web/PostgreSQL disponibles ; shared non isolé	run CI du SHA final et rapports bruts archivés
C2.2.3	Correctifs sécurité, conformité métier et accessibilité en cours	revue OWASP + audit RGAA manuel/auto
C2.2.4	Git et production existent, mais ne ciblent pas encore ce correctif	merge, tag, déploiement et smoke du même SHA
C2.3.1	Inventaire reconstruit ; recette finale non exécutée	exécution traçable de toutes les fonctions
C2.3.2	Registre enrichi avec les défauts réellement découverts	clôture liée aux tests de non-régression
C2.4.1	Manuels présents ; procédures non validées depuis un clone vierge	test depuis un clone vierge et paquet final

Cette table ne préjuge pas de la décision du jury. Elle distingue volontairement le code écrit d'une preuve d'exécution effectivement obtenue.

3. C2.1.1 — Environnements, qualité et performance

Environnement de développement et de test

Élément	Choix du projet	Vérification
Gestion de sources	Git et dépôt GitHub	branche, SHA, historique, PR/CI
Gestion monorepo	pnpm workspace	<code>pnpm install --frozen-lockfile</code>
Runtime de référence	Node.js 24 LTS en local, CI, conteneurs et Vercel	<code>node --version</code>
Compilateur	TypeScript 5.7 via <code>tsc</code>	<code>pnpm typecheck</code> , <code>pnpm build</code>
Serveur Web	Next.js 15 App Router	<code>build</code> et <code>healthcheck</code> Web
Serveur API	Hono sur Node.js	tests routes et <code>health/readiness</code> API
Base	PostgreSQL 16, Drizzle ORM	migrations et tests d'intégration
Tests	Vitest, Testing Library, Playwright, axe	rapports API, Web et navigateur
Conteneurs	Docker multi-stage et Compose	<code>build</code> , <code>migration/seed</code> , <code>healthchecks</code>
Production	Vercel Web/API et Neon PostgreSQL	déploiement et smoke tests

Le candidat doit ajouter dans la version remise le nom et la version de l'éditeur effectivement utilisé. Le présent audit a été réalisé sous Windows, PowerShell et Codex desktop ; cette information ne doit pas être remplacée par un éditeur fictif.

Gates qualité visées

La liste ci-dessous décrit les conditions de fermeture. Elle ne signifie pas qu'elles ont déjà toutes été exécutées sur un SHA final :

- aucune erreur ESLint ;
- aucune erreur TypeScript ;
- tests API et Web réussis, contrats shared exercés et périmètre shared explicitement documenté ;
- rapport de couverture API et Web conservé sans exclusions opportunistes ;
- build de production réussi sous Node 24 ;
- audit high/critical bloquant ;
- Playwright public et authentifié ;
- build Docker et procédure de migration/seed exécutables.

Performance

Un simple HTTP 200 ne constitue pas une mesure de performance. La preuve finale doit au minimum conserver : durée build/CI, latence des healthchecks, durée des générations IA, timeout observé, taille des pages ou Web Vitals et résultat d'un petit test de charge sur les routes sans coût IA. Les objectifs doivent être reliés à l'usage : navigation réactive, absence de 504 et retour d'erreur clair avant la limite Vercel.

Une première mesure locale réelle est consignée dans B2-A21 : sur 50 requêtes séquentielles, le p95 observé est de 8,18 ms pour `/health` et 36,35 ms pour `/health/ready`. Elle couvre PostgreSQL local, mais ni Vercel/Neon ni OpenAI, et ne remplace donc pas la mesure du SHA final en production.

4. C2.1.2 — Intégration continue

Le protocole cible est :

1. checkout du SHA ;
2. installation figée par lockfile ;
3. build du package partagé ;
4. lint et typecheck ;
5. tests et couverture API/Web ;
6. build des packages ;
7. tests Playwright et accessibilité ;
8. audit de dépendances high/critical bloquant ;
9. build des images Docker ;
10. déploiement uniquement après une CI verte sur main.

Le workflow CD manuel qui permettait de contourner la CI est supprimé de la version candidate. Le déploiement reste conditionné par la variable de projet et par un `workflow_run` réussi. La preuve finale sera un run GitHub du SHA remis, pas le run historique 29489995458.

5. Architecture maintenable

Navigateur

- > Next.js Server Components / Server Actions
- > API Hono protégée par secret interservice et identité utilisateur
- > controllers -> services métier -> repositories
- > PostgreSQL / OpenAI

Les contrats partagés sont placés dans packages/shared. Les entrées HTTP et les sorties IA sont validées par Zod. Les accès aux ressources sont contrôlés avec l'identité de l'utilisateur. Les corrections de cette version introduisent notamment une couche service pour les journaux de séance afin de ne plus faire porter l'ownership au seul contrôleur.

Écarts architecturaux restant à suivre : duplication de certains schémas d'entrée entre API et shared, observabilité fondée sur console.* et rate limit mémoire non distribué.

6. C2.2.1 — Prototype et besoins couverts

Le prototype vise un utilisateur sportif authentifié et couvre les user stories suivantes :

ID	Besoin attendu	Parcours
US-01	Se connecter et protéger les données personnelles	OAuth Google, routes privées
US-02	Générer une séance adaptée	/generate puis détail
US-03	Générer un programme multi-semaines cohérent	/programs/generate puis détail
US-04	Retrouver et filtrer ses séances/programmes	listes, filtres, pagination
US-05	Exécuter une séance avec pause/reprise	Timer
US-06	Journaliser effort, feedback et douleur éventuelle	fin de séance
US-07	Suivre sa progression	dashboard
US-08	Choisir le modèle OpenAI autorisé	settings
US-09	Supprimer une ressource avec confirmation	dialogues accessibles
US-10	Utiliser les parcours au clavier et sur mobile	audit RGAA représentatif

Le prototype de référence demeure <https://ai-sport-web.vercel.app>, mais il ne sera une preuve de la version candidate qu'après déploiement du SHA final.

7. C2.2.2 — Harnais de tests unitaires

Le harnais comprend :

- tests des schémas et invariants métier partagés ;
- tests des services IA, erreurs, retry et timeout global ;
- tests controllers et validation UUID ;
- tests d'ownership pour workout, programme et journaux ;
- tests Web de logique, composants et erreurs utilisateur ;
- tests d'intégration PostgreSQL ou preuve explicitement séparée ;
- tests Playwright, qui complètent mais ne remplacent pas les unitaires.

Les rapports API et Web sont publiés séparément. La mesure locale du 2026-07-20 sur la candidate 0.13.0-rc.1 donne :

Rapport	Statements	Branches	Functions	Lines	Périmètre/exclusions
API unitaire	84,97 %	80,40 %	95,38 %	84,97 %	src, hors bootstrap, DB, repositories et routes déclaratives ; repositories mesurés séparément en intégration PostgreSQL
API intégration PostgreSQL	93,69 %	80 %	100 %	93,69 %	repositories et service d'ownership inclus par <code>vi test.integration.config.ts</code> ; 8 tests réels sur PostgreSQL 16.14
Web	68,07 %	77,45 %	79,38 %	68,07 %	app, composants, lib ; les pages serveur non instanciées apparaissent bien à 0 %
Shared	Non isolé	Non isolé	Non isolé	Non isolé	schémas exercés par 6 tests de contrats API, mais pas de rapport instrumenté autonome

Les suites locales comptent 86 tests API et 39 tests Web réussis ; leurs sorties ont été observées pendant la correction mais ne sont pas encore archivées comme annexes brutes du SHA final. Un PostgreSQL 16.14 réel a également exécuté 8/8 tests d'intégration sur la candidate locale 69b21ef-dirty. Ce rapport reste séparé du rapport unitaire et est consigné dans B2-A19 ; il doit encore être rejoué en CI sur le SHA final. Les 48 cas Playwright publics ont aussi réussi localement, sans rapport brut final archivé, et ne sont pas comptés comme tests unitaires.

Le nombre de tests ou un pourcentage API isolé ne suffit pas. Cette mesure montre une majorité sur les périmètres instrumentés, avec une faiblesse visible sur plusieurs pages serveur Web. La clôture C2.2.2 reste donc conditionnée au run CI du SHA final et à l'archivage de tous les rapports bruts.

8. C2.2.3 — Conformité fonctionnelle, sécurité et accessibilité

Conformité fonctionnelle

Les sorties IA ne sont plus acceptées uniquement parce que le JSON est valide. Les schémas et services vérifient les invariants utiles : durée, structure, numérotation et quantité de semaines/séances. Les cas force/répétitions et les tolérances retenues doivent rester couverts par des tests métier.

Le Timer mesure le temps actif et non le temps mural incluant les pauses. Les erreurs réseau ne sont plus transformées silencieusement en fausses 404 ou en paramètres prétendument enregistrés.

Sécurité OWASP Top 10

La revue détaillée se trouve dans docs/security/owasp-review.md. Les preuves ne doivent pas réduire l'OWASP Top 10 à npnm audit. Les points centraux sont :

- contrôle d'accès et ownership de chaque ressource liée ;
- secrets utilisés dans les modules serveur ; l'absence dans les bundles et le réseau de la candidate déployée reste à vérifier ;
- requêtes Drizzle paramétrées ;
- validation des entrées et des IDs ;
- timeout global inférieur à la limite de la fonction ;
- configuration CSP/CORS/headers ;
- audit de dépendances bloquant ;
- intégrité de la CI et versions d'outils figées ;
- logs/monitoring et traitement des incidents ;
- URL OpenAI fixe pour prévenir la SSRF.

Les risques résiduels, notamment rate limit mémoire et absence de SIEM, sont présentés comme tels et non comme des contrôles complets.

Accessibilité

Le référentiel choisi est le RGAA 4.1.2, fondé sur WCAG 2.1 A/AA. Le choix, l'échantillon et la méthode figurent dans docs/rncp/bloc2-accessibilite-rgaa.md.

La recette instrumentée locale B2-A20 a réussi 12/12 contrôles sur Chromium et 12/12 sur Firefox. Elle couvre quatre pages publiques à 320 px, axe ciblé, console et erreurs JavaScript, le lien d'évitement, le nom du bouton Google et quatre redirections sans session. Elle a conduit à corriger deux défauts de focus/nom accessible et un contraste visuellement faible. /dashboard, OAuth, la suite authentifiée et l'audit RGAA manuel n'ont pas été exécutés dans cette preuve. Les contrôles humains complets — zoom, ratios de contraste, lecteur d'écran, dialogues, onglets et Timer — restent à consigner. Le statut « conforme RGAA » reste interdit tant que l'audit humain final n'est pas terminé.

9. C2.2.4 — Version, déploiement et viabilité

La version finale doit suivre cette séquence :

1. intégrer les corrections sur main ;
2. exécuter toutes les gates ;
3. relever le SHA immuable du commit validé ;
4. appliquer les migrations tracées ;
5. déployer exactement ce SHA ;
6. vérifier readiness, login et parcours métier ;
7. recueillir un test d'utilisation autonome ;
8. créer le tag sur ce SHA seulement après ces vérifications ;
9. joindre les preuves et renseigner le manifeste.

Le changelog distingue les versions historiques, les évolutions Unreleased et la future release de correction. Les anciens domaines alcide-* ne sont plus des cibles de production.

10. C2.3.1 — Cahier de recettes

Le cahier de recettes est `docs/bloc2/cahier-recettes.md`. Son inventaire couvre les familles fonctionnelles identifiées : authentification, séances, programmes, listes, détail, suppression, Timer, journaux, dashboard, paramètres, sécurité, accessibilité, healthchecks, CI/CD et déploiement.

Pour chaque cas, il distingue :

- attendu ;
- obtenu ;
- environnement et date ;
- exécution manuelle, automatique ou inspection ;
- preuve ;
- anomalie associée.

Une inspection du repository ou le comportement supposé de React/Zod n'est pas considéré comme une recette exécutée.

11. C2.3.2 — Correction des bogues

Le registre se trouve dans `docs/rncp/bloc2-plan-correction-bogues-rncp39583.md`. Les défauts découverts le 2026-07-20 y sont traités comme de vraies anomalies : couverture partielle, fixture E2E vide, ownership session-log, invariants IA, Timer, accessibilité, CSP, Docker, CI sécurité, versionnement et incohérences documentaires.

Une anomalie ne passe à « corrigée » qu'après correctif, test de non-régression réussi et preuve rattachée au SHA final.

12. C2.4.1 — Documentation d'exploitation

Document	Rôle
docs/deployment.md	déploiement Vercel, Neon et Docker
docs/ci-cd.md	séquences CI/CD et rollback
docs/rncp/bloc2-manuel-utilisateur-alcide.md	parcours et erreurs utilisateur
docs/rncp/bloc2-manuel-mise-a-jour.md	évolution, migrations, rollback
docs/security/owasp-review.md	revue sécurité et risques résiduels
docs/rncp/bloc2-accessibilite-rgaa.md	référentiel et audit accessibilité
docs/rncp/MANIFESTE-DEPOT-BLOC2.md	contenu et identité de la remise

La procédure Docker utilise des services migrate et seed basés sur le stage builder. Elle ne demande plus d'exécuter drizzle-kit ou tsx dans l'image API de production qui ne les contient pas.

B2-A22 consigne l'exécution locale réelle des deux builds Node 24/pnpm 11.9, des runtimes non-root, de migrate, de seed et du nettoyage ciblé. Le contrôle depuis un clone vierge et le job Docker du SHA final restent à produire.

13. Annexes à produire sur la version finale

- sortie brute lint/typecheck/build ;
- résultats et rapports de couverture API/Web/shared ;
- résultats des tests PostgreSQL ;
- rapport Playwright public/authentifié avec traces en cas d'échec ;
- rapport axe sans filtrage des seules violations graves ;
- grille d'audit manuel RGAA ;
- audit de dépendances ;
- run CI et CD du SHA final ;
- healthchecks et parcours post-déploiement ;
- captures desktop/mobile ;
- retour d'un utilisateur autonome ;
- manifeste avec archive, SHA, tag et version.

14. Conclusion

Alcide dispose d'un prototype 0.12.0 historiquement déployé et d'une candidate locale 0.13.0-rc.1. La candidate traite des défauts constatés lors de l'audit du 2026-07-20 qui n'étaient pas explicités dans le dossier du 2026-07-16. Elle ne doit toutefois pas être annoncée comme « validable » tant que les champs À RENSEIGNER, la recette complète, l'audit RGAA, la CI/CD et le déploiement du même SHA ne sont pas fermés.

Cette formulation vise à distinguer ce qui est implémenté, ce qui est prouvé et ce qui reste à exécuter, sans abaisser les critères officiels des compétences éliminatoires.